



Taller de Integración y Despliegue Continuo (CI/CD)

Fundación Universitaria Tecnológico Comfenalco

Ingeniería de Software · Taller de CI/CD

INTEGRANTES

ALISON ATENCIO RAMOS

JESUS EDUARDO JULIO ACOSTA

JUAN JOSÉ HERNÁNDEZ BASSA

Taller de Integración y Despliegue Continuo (CI/CD)

Proyecto **maddelivery** (Spring Boot 3.5.6 · Java 21) — Reporte de evidencias por cada paso del pipeline — Repositorio: **Proyecto Actualizado**

1. Resumen del pipeline implementado

Etapa		Acción implementada	Resultado del paso
Paso 1 — Integración continua (CI)		GitHub Actions <code>.github/workflows/ci.yml</code> : compila, ejecuta pruebas y genera reporte de cobertura en cada push/PR a main y develop.	Verificado
Paso 2 — Build de la aplicación		Maven <code>clean compile</code> con JDK 21 (Temurin) y empaquetado del <code>.jar</code> ejecutable.	BUILD SUCCESS
Paso 3 — Pruebas unitarias		Suite de pruebas del carrito de compras: <code>CartServiceTest</code> y <code>CartControllerTest</code> .	11/11 OK
Paso 4 — Cobertura de código		Plugin JaCoCo 0.8.12 (<code>prepare-agent + report</code>). Reporte publicado como artefacto <code>reporte-jacoco</code> .	Generado
Paso 5 — Despliegue/CD & secretos		Secreto <code>APP_ENV_DEMO</code> (variable de entorno protegida) usado en la etapa final del flujo.	Configurado

2. Evidencia del Paso 2 — Build local (Maven)

Comando: `mvn clean verify` (JDK 21 · Maven 3.9.9 · Spring Boot 3.5.6)

```
[INFO] --- maven-resources-plugin --- (default-testResources) ---
[INFO] --- jacoco:0.8.12:report (report) @ maddelivery ---
[INFO] Loading execution data file ...target\jacoco.exec
[INFO] Analyzed bundle 'maddelivery' with 19 classes
[INFO] --- jar:3.4.2:jar (default-jar) @ maddelivery ---
[INFO] Building jar: ...target\maddelivery-0.0.1-SNAPSHOT.jar
[INFO] --- spring-boot:3.5.6:repackage (repackage) @ maddelivery ---
[INFO] Replacing main artifact ...maddelivery-0.0.1-SNAPSHOT.jar with repackaged archive
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 01:04 min
[INFO] Finished at: 2026-09-02T17:39:06-05:00
```

Artefacto generado: `maddelivery-0.0.1-SNAPSHOT.jar` (**77.5 MB**, jar ejecutable con dependencias anidadas en `BOOT-INF/`).

3. Evidencia del Paso 3 — Pruebas unitarias

Tests run: 11, Failures: 0, Errors: 0, Skipped: 0

Clase de prueba	Ámbito	Tests	Fallos	Errores	Tiempo
<code>CartControllerTest</code>	Endpoints del carrito (REST/Controller)	6	0	0	16.64 s
<code>CartServiceTest</code>	Lógica de negocio del carrito (Servicio)	5	0	0	1.27 s

Salida del log Maven (Surefire)

```
[INFO] Running com.maddelivery.maddelivery.controlador.CartControllerTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 16.64 s
[INFO] Running com.maddelivery.maddelivery.servicio.CartServiceTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.270 s
[INFO] Results:
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
```

4. Evidencia del Paso 4 — Cobertura de código (JaCoCo)

Plugin: jacoco-maven-plugin 0.8.12 con ejecuciones prepare-agent (fase init) y report (fase test). Reporte HTML en target/site/jacoco/index.html, publicado como artefacto reporte-jacoco.

19

CLASES ANALIZADAS

10

INSTRUCCIONES

Cobertura de las clases con pruebas unitarias

Clase	Líneas cubiertas	Instrucciones cubiertas %
CartServiceImpl	30 de 31	93.7%
CartController	10 de 11	87.0%
CartEntity	5 de 5	100%

El reporte completo navegable se encuentra en maddelivery/target/site/jacoco/ y como artefacto de GitHub Actions (reporte-jacoco).

5. Evidencia del Paso 1 — Pipeline en GitHub Actions (.github/workflows/ci.yml)

```
name: CI Pipeline Spring Boot

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout del código
        uses: actions/checkout@v4
      - name: Configurar JDK 21
        uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: 'temurin'
          cache: maven
      - name: Build con Maven
        run: mvn -B clean compile
        working-directory: maddelivery
      - name: Ejecutar pruebas unitarias
        run: mvn -B test
        working-directory: maddelivery
      - name: Reporte de cobertura JaCoCo
        run: mvn -B jacoco:report
        working-directory: maddelivery
      - name: Publicar reporte como artefacto
        uses: actions/upload-artifact@v4
        with:
          name: reporte-jacoco
```

```
    path: maddelivery/target/site/jacoco
- name: Verificar variable de entorno
  env:
    APP_ENV: ${ secrets.APP_ENV_DEMO }
  run: echo "Ejecutando en el ambiente configurado"
```

6. Evidencia del Paso 5 — Configuración de secretos

En GitHub (Settings → Secrets and variables → Actions) se define el secreto **APP_ENV_DEMO** (valor de ejemplo: workshop-2026). El pipeline lo consume con `${ secrets.APP_ENV_DEMO }` y lo expone como variable de entorno **APP_ENV** en la etapa final. Los secretos nunca se registran en el log de la acción.

7. Control de versiones (Git)

Hash	Mensaje del commit
3b4d2f6	chore: pipeline CI/CD con GitHub Actions, tests unitarios, cobertura JaCoCo, .gitignore y limpieza de credenciales
d49e508	Primer commit del proyecto actualizado

Rama por defecto: main · Remoto: <https://github.com/Mauro2513/proyectoactualizado.git> · .gitignore agregado para excluir target/, .idea/ y credenciales.